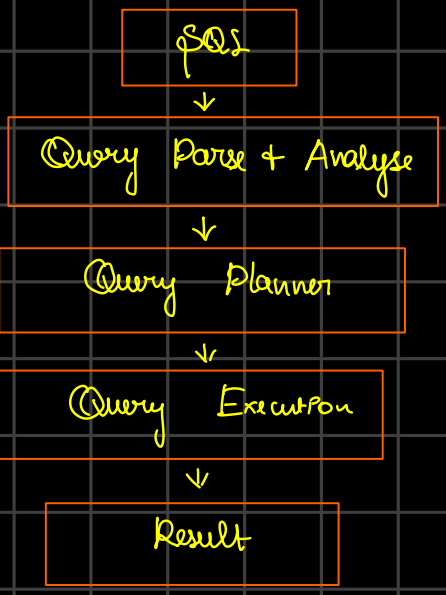

PostgreSQL

What is PostgreSQL ?

* PostgreSQL is an relational database. That used to store data in the table format.
* It is a software to manage the database.

How it works



SQL - The Actual Raw Query.

Parsing - Used to check the Gramatical and the structure of the Query. Just need to check whether it's a valid Query. It uses Parse tree.

Semantic Analyser - whether table, name column are all valid

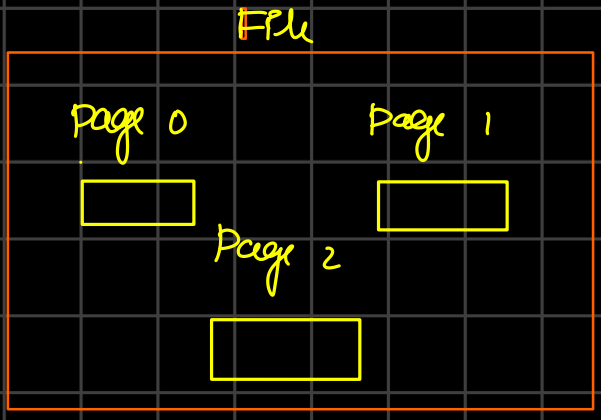
Query Planner - There is lot of ways to get the result- which makes cost less and efficient with the sequential scan or Index based search

Result - The actual result in table.

How PostgreSQL stores data

* PostgreSQL used to store the data in binary format in a file since its store in the disk or RAM

* In a file it is used to speared a fixed size of section like 8KB is called Pages



* Each Page contains many rows and a single row is called Tuple

* Chunking the data used to help for direct page searching using Index and that also used to put in RAM for the faster Execution

* When we look the data in the file of PostgreSQL which looks in gibrish.

Transcation
=

This is the Concept of the database System a group of one or more Operation that treated as a Single Unit of work.

ACID
=

ACID is a set of four Properties of the transcation System works

Atomicity - It means "All or Nothing" this done with the help of BEGIN, COMMIT and ROLLBACK

```
BEGIN;
UPDATE account SET balance = balance - 500 WHERE id=1
UPDATE account SET balance = balance + 500 WHERE id=2
COMMIT;
```

If any one Upatation fails its rollback or undone to the previous State.

* When transcation BEGINS the DBMS used to Create an transcation ID for the Operation (XID)

* When Making Changes used of directly Updating row that used to over the Current row snapshot and the up dated row data. Every Changes are written into the Write Ahead Log (WAL) instead of directly Changing the record

(WAL) a Sequential Changes of logs.

* When you started committing it used to write (XID) this transcation is committed marked into the (WAL) and after the XID 5000 is committed and after this record marked COMMITTED SUCCESSFULL

* When Rolled Back. It does not do commit record instead it mark XID 5000 is aborted and nothing Record Changes all the newly recorded Version is invisible and the Junk record will be Cleared with the help of VACCUM

Consistency - * The current database is in Valid State and after the transcation that also must be valid that what called Consistency means. It should not Violate the database state.

* With the help of the "Constraints" in the transcation enforces the state to be Valid

Isolation - Its Make each transcation Isolated even if Concurrency transcation Processing.

Durability - Since every thing Processing and Concurrently working When Power loss or Crashed (Server) Since (WAL) works in Ram so before Commit record marked it sync up with the disk to Prevent loss of memory.

MVCC Version Concurrency Control

* Traditional Database using lock system for concurrency but Postgres uses the Multiple version of the row coping and logging into the (Write Ahead Log)

* The Visibility of the Version is the Heart of the MVCC that have some set of Rules to show the Row Version.

Index

* Index is a special data structure that used to Search faster instead of Sequential Scan

* We can use many types of Index but by default B-Tree Index is default.

B-Tree Indexing

Btree is Balanced tree so the depth of each leaf node is same level

Root Node - Contains Meta data and Information
Internal Node - Contains Key and Children node Pointer
Children Node - Actual [Indexed Value , TID]

Left Value < Right Value so Order and Search are faster O(log N) time.

If we Indexed every Column every Column Creates Btree so more storage Occupied and Insert / update / Delete much slower because need to every in tree. more Vaccum Process

Hash Index - It store in Key Value data structure for Example Key. The value we search and TID (Pages, Row)

hash(ab) - TID(5,3) ...
hash(xy) - TID(8,5) ...

GIN Index (Generalized Inverted Index)

Rows -> List of Values
Values -> list of Rows

Pid	Tag
1	{apple, orange}
2	{apple, banana}
3	{orang, banana}

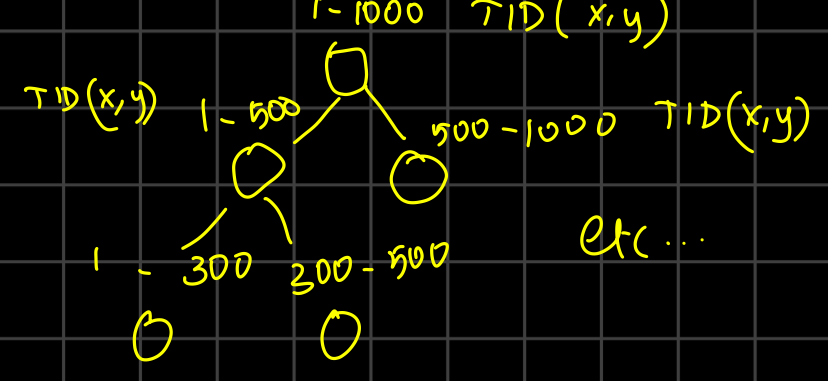
GIN Index

Key: Apple TID (1,1) (2,1)
Key: banana TID (2,2)(3,2)
. . . . .

Good for list of Values or Text like array Jsonb etc...

GIST Index - A flexible Frame work used for Complex data. nodes have bounds and ranges to stored

(ranges can Overlap)



SP- GIST - Space Partitioned GIST

Unlike GIST it partition the node which is non overlapping

Block Range Index - BRIN

It used to store the Block of Ranges Instead of storing all Values

Block Range 0 (0 - 127)

Block Range 1 (128 - 225) . . . .

Just store the block of range

Normalization

Normalization is the Process of Organizing the data to prevent unnecessary duplication and Prevent data Related Problem!

Id	Name	Course	Instructor
1	Arun	postgres	Ravi
2	Ram	Java	Amit
3	Arun	Java	Amit
4	Ram	postgres	Ravi

When Need to change Ravi to Rahul For postgres Need to update many Row so efficient Reduced

Normalization Helps Problem

Separate tables

student

Pid	Name
1	Arun
2	Ram

Pid	Course	Instru
1	Ravi	Postgres
2	Amit	Java

How its solve the things for efficiency.

Types of Normalization Called "Norm Forms"

1NF - First Normal Form which is every record Contains a Single Value No Repetation or No Multiple Values

Ravi	maths, Science, Social

(Bad)

Ravi	maths
Ravi	Science
Ravi	Social

(Good)

2NF - Second Normal form. That is must be in the 1NF and No Partial dependency (No more than one Column depend). Every Non key column must depend on Entire Primary Keys.

"No Partial Deps"
(Every Non Key Column Must Dep on Entire PK)

StdPid	Subject	StdName	Marks
1	Maths	Ravi	90
1	Scienc	Ravi	85
2	Maths	Priya	92

(Student Pid + Subj) becomes Primary Key

(Bad)

StdId	Name
1	Ravi
2	Priya

StdId	Subject	marks
1	Maths	90
1	Scienc	85
2	Maths	92

(Good)

3NF - Third Normal Form. A Non Key Column should not depend on another Non Key Column. No Transitive dependency.

Empid	Emp Name	depid	depname
1	Ravi	101	Engineering
2	balu	101	Engineering
3	priya	102	Marketing

Empid → depid → depname (which is transitive)

Empid	Emp Name	depid
1	Ravi	101
2	balu	101
3	priya	102

depid	depname
101	Engineering
101	Engineering
102	Marketing

Now the Transitive Broken.

BCNF - Boyce Codd Normal Function.

* If one Column determine another Column the determinant must be a super key.

Student	Subject	Instructor
Arun	SQL	Ravi
Bala	SQL	Ravi
Charan	Java	Kumar

Our rule each teacher teaches only one subject but a teacher colour cannot identify the single row because single teacher can teach to many Student

so decompose (Split the table)

Student	Instructor
Arun	Ravi
Bala	Ravi
Charan	Ku

Subject	Instructor
SQL	Ravi
SQL	Ravi
Java	Kumar

4NF - Forth Normal form

* Multivalued dependency
* It means our things can have multiple values
* Student Column and Hobbies Column a Student can have multiples hobbied so many duplicated values

Student	Hobby	Language
Arun	Cricket	Tamil
Arun	Cricket	English
Arun	Chess	Tamil
Arun	Chess	English

two Independent Columns

Student	Hobby
Arun	Cricket
Arun	Chess

Student	Language
Arun	Tamil
Arun	English

5NF - Split a table into small tables and combine them when its Needed

6NF - Split the tables

GRANT + REVOKE Commands in SQL

GRANT - Used to Give Permission to the action

Imagine we have User Named "Rahim"

GRANT SELECT * From Employes. Only Read Permission

REVOKE - Used to remove the Permission

REVOKE INSERT ON employees TO rahim ;

(Just this Command used to Provide and remove Permissions)

Common Table Expression - CTE

* Its an advanced Concept of the Subqueries
* Its like storing a sub Query and name it then using it later.

```
WITH Topper AS
( Select * From students
  where total_mark > 450)
Select * From Toppers ;
```

(Single Table Stored)

```
WITH Topper AS
( Select * From Students
  where total_mark >450)
    Duller AS
( Select * From Student
  where total_mark < 200)
Select topper.name As topper,
       Duller.name As Duller,
FULL JOIN roper.Pd = duller.Pd;
```

(Multi table Stored)

Recursion using CTE

```
WITH RECURSION AS Sum_n (Here in the bracket you can mention
                         that refers the fuithayer name)
                                  Eg : Sum_n (n)
( Select 1 As n -> Initialization
  UNION All
  Select n + 1
  From Sum_n
  where n ≥ n)
Select * From numbers ;
```

WINDOW Function

It makes Calculation for a selected or a group of row and give the result in each row

"The WINDOW is a Viewing PORT"

Syntax

```
function_name ()
over ( partition by column_name
       order by column_name
     )
```

over() tell us is the window function
function_name the Calculation you need to do

Types of WINDOW

several build in function and can be used with Aggregated function.

```
Select StudentName, Row_number() over() From Studen
Select Avg(total_marks) over( order by total_mark) From Studj
```

View

View is used to store Query and can use it later in anywhere

```
CREATE VIEW as Old_man
Select name From People
Where age >= 60 && Gender = male
```

} Saved The Query Not Result and Query The result While Calling it

Materialized VIEW

Unlike Computing Everytime Its used to store the result itself on disk Refersh Materialized View Maintance to reflects the data after Any insertion or update happen

Full Search Text

Unlike the LIKE or ILIKE it used to search efficiently and more advanced features it have

to_tsvector('english', 'Learning the Postgres') - this function used to convert this words into searchable words like :- database:2 , 'learn':1 , 'use:4'

to_tsquery('english', 'Learning & database') - This function used to perform search Query

```
SELECT * From Articles
where to_tsvector('english', title)
@@ to_tsquery('english', 'Learning & database')
```

If the id found it's update the value else it update

```
Insert into Employee values (1, 'Arun', 4000)
ON CONFLICT (id)
DO UPDATE SET Salary = EXCLUDED.salary.
```

```
Insert into Employee values (1, 'Arun', 4000)
ON CONFLICT (id)
DO NOTHING;
```

SET Theories

UNION = JOIN tables Vertically

UNION All = With Duplicates

INTERSECT = Pick up Common Things

EXCEPT = Picks first Query Data that's not in the second Query.

Syntax

```
SELECT id, name From students1
UNION
SELECT id, name From students2;
```

Can be Any set value

Stored Procedure

* This Stored Procedure is used to do some Programming in the database.

* Sometime Called as Procedure Language, PL, PL pgSQL ....

How It's work

When see a function that used to register the Function in the PG-Catalog (like meta data) and when Calling the actuall Execution starts with the Help of Query Parser, Query Planner, Executer.

Syntax

Create Function functionName - Used to create fn

Create or Replace Function functionName - Create or Replace

$$ - Kind of { } this function body starts

DECLARE - using this we can declare variable

BEGIN - Start point of code Logics

END; - End point of code Logics

Language PLpgSQL - define which language to use because PG used multiple language like Extension language

Here is the Example

```
CREATE OR REPLACE FUNCTION get_emp_details (
    emp_id INT) -  parameter
RETURN TEXT  - Return type
LANGUAGE PLPGSQL
AS $$
DECLARE
    emp_name TEXT;
    emp_salary NUMERIC;
    result TEXT;
BEGIN
    SELECT name, salary
    INTO emp_name, emp_salary
    FROM employees
    WHERE id = emp_id;

    IF NOT FOUND THEN
     RETURN "Employee NOT found"
    END IF;
    RETURN emp_name || emp_salary
END;
$$;
```

Calling Function

```
SELECT get_emp_details (10028);
```

Deleting Function

```
DROP get_emp_details ();
```

Triggers

* A trigger that used to automatically execute when some event happened.

* The trigger can fire before the event fires or after the event fires with the help of BEFORE & AFTER keyword.

Examples:

* Trigger when post updated - AFTER
* Trigger before search Text - BEFORE

Types of Triggers

ROW level Triggers - For each row the triggers run "FOR EACH ROW"

Statement level Trigger - The trigger run only once per statement. "FOR EACH STATEMENT"

Syntax -

1.) First Need to Create trigger function

```
CREATE OR REPLACE FUNCTION record_salary_change()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS $$
BEGIN
        INSERT INTO Salary_audit
        VALUES (OLD.Id, OLD.salary, NEW.salary);
        RETURN NEW;
END;
$$
```

2) Create trigger

```
CREATE TRIGGER salary_change_trigger
AFTER UPDATE OF salary ON employees
FOR EACH ROW
EXECUTE FUNCTION record_salary_change();
```